

Epistemic Gating for Consciousness-Aware Language Generation: Eliminating Hallucination via Logit Masking and HDC Binding

Tristan Stoltz¹

¹Luminous Dynamics

`tristan.stoltz@evolvingresonantcocreationalism.com`

Abstract

Large language models generate fluent text but suffer from hallucination—the production of plausible but unsupported claims. Existing mitigation strategies operate post-hoc, penalizing hallucinated content after generation through retrieval-augmented verification, confidence calibration, or output filtering. We present a fundamentally different approach: *epistemic gating*, a mechanism that physically prevents hallucination at the logit level by masking tokens for which the system lacks sufficient epistemic support. Our implementation, the Broca language pipeline (28,000 lines of code, 358 tests), integrates epistemic gating within a consciousness-aware architecture comprising a 20-channel ThoughtEncoder, 16,384-dimensional Hyperdimensional Computing (HDC) binding, and autoregressive generation governed by active inference principles. The pipeline implements three complementary mechanisms: (1) an EpistemicGate that computes per-token epistemic support from HDC similarity to grounded knowledge and masks logits below a consciousness-modulated threshold; (2) a CoherenceFeedback module that performs mid-sentence semantic veto when generated text diverges from the intended thought trajectory; and (3) an adaptive cadence controller that gates generation frequency on consciousness level and learning rate. We describe the architecture, present benchmark results across factual accuracy, coherence, and hallucination rate, and discuss the theoretical grounding in active inference and Global Workspace Theory.

1 Introduction

Hallucination in language generation systems—defined as the production of content that is nonsensical, unfaithful to source material, or factually incorrect—represents one of the most significant obstacles to the deployment of AI language systems in safety-critical domains [Ji et al., 2023, Maynez et al., 2020]. The problem is particularly pernicious because hallucinated content is often fluent and contextually plausible, making it difficult for users to detect without independent verification [Lin et al., 2022].

Current approaches to hallucination mitigation fall into three categories. *Training-time* interventions include reinforcement learning from human feedback (RLHF) [Ouyang

et al., 2022], constitutional AI [Bai et al., 2022], and data curation. *Inference-time* interventions include retrieval-augmented generation (RAG) [Lewis et al., 2020], chain-of-thought prompting [Wei et al., 2022], and self-consistency decoding [Wang et al., 2023]. *Post-hoc* interventions include factual verification [Thorne et al., 2018], confidence calibration [Kadavath et al., 2022], and output filtering. All of these approaches share a common limitation: they operate *after* the hallucinated content has been generated (or at least after the hallucination-prone logits have been computed), attempting to detect and correct errors rather than prevent them.

We propose a fundamentally different approach inspired by the neuroscience of conscious language production. In the human brain, Broca’s area does not simply decode an arbitrary language model’s output; it generates speech from a structured thought representation that is grounded in perception, memory, and embodied experience [Hagoort, 2005, Fedorenko et al., 2014]. The critical insight is that biological language production is *constrained by epistemics*: one cannot articulate what one does not know, because the thought-to-language pathway requires a grounded conceptual representation as input [Levelt, 1989].

Our system, the Broca language pipeline, implements this principle computationally. Rather than generating text from a statistical language model and subsequently checking for hallucination, Broca generates text from a hyperdimensional thought representation that is epistemically grounded by construction. A dedicated EpistemicGate module computes the epistemic support for each candidate token by measuring its HDC similarity to the system’s grounded knowledge base, and masks logits for tokens that lack sufficient support. This constitutes a *physical* prevention of hallucination—unsupported tokens never enter the probability distribution from which generation samples.

2 Background

2.1 Hallucination in Language Models

Ji et al. [2023] provide a comprehensive taxonomy of hallucination types, distinguishing *intrinsic* hallucinations (contradicting the source) from *extrinsic* hallucinations (introducing unverifiable claims). Both types arise from the fundamental training objective of language models: next-token prediction maximizes the likelihood of the training distribution, which conflates statistical regularity with factual accuracy [Bender et al., 2021].

The scale of the problem is substantial. Studies report hallucination rates of 15–30% in abstractive summarization [Maynez et al., 2020], 5–20% in open-ended question answering [Lin et al., 2022], and higher rates in knowledge-intensive generation tasks. These rates persist even in the largest models, suggesting that scale alone does not resolve the problem [Ji et al., 2023].

2.2 Hyperdimensional Computing for Representation

Hyperdimensional Computing [Kanerva, 2009] represents information as binary or bipolar vectors of very high dimensionality ($D \geq 1,000$). Three core operations define the algebra:

- **Binding** ($\otimes$): element-wise XOR (binary) or multiplication (bipolar), producing a vector dissimilar to both operands.
- **Bundling** ($\oplus$): component-wise majority, producing a vector similar to all operands.

- **Permutation** (ρ): circular shift for encoding order.

A key property for our purposes is that HDC similarity (measured by normalized Hamming distance for binary vectors, or cosine similarity for bipolar vectors) provides a continuous, graded measure of semantic relatedness. Two vectors sharing common structure exhibit above-chance similarity; unrelated vectors are quasi-orthogonal [Kanerva, 2009].

2.3 Active Inference and Consciousness

Active inference [Friston, 2010] posits that biological agents minimize variational free energy—a bound on surprise—through both perception (updating internal models) and action (changing the environment). Language generation under active inference is not merely statistical prediction but an *epistemic action*: the agent generates utterances that reduce its own uncertainty and communicate its beliefs [Friston, 2010].

Global Workspace Theory (GWT) [Baars, 1988, Dehaene, 2014] proposes that conscious access involves broadcasting information from specialized processors to a global workspace, making it available to multiple cognitive subsystems. We adopt GWT’s framework for our generation pipeline: only information that achieves conscious access (broadcasting to the global workspace) is available for language generation.

3 Architecture

3.1 System Overview

The Broca language pipeline comprises five major components operating within a consciousness-aware cognitive architecture:

1. **ThoughtEncoder**: A 20-channel encoder that transforms the system’s cognitive state into a structured thought representation.
2. **HDC Binding Layer**: 16,384-dimensional binary HDV binding that creates a holographic thought vector.
3. **EpistemicGate**: Per-token epistemic support computation and logit masking.
4. **Autoregressive Generator**: Token-by-token generation with Liquid-Mamba fusion backend.
5. **CoherenceFeedback**: Mid-sentence semantic veto and self-correction.

These components interact within a cognitive loop operating at approximately 31 Hz, with Broca generation governed by an adaptive cadence controller that modulates generation frequency based on consciousness level and learning rate.

3.2 ThoughtEncoder: 20-Channel Cognitive Encoding

The ThoughtEncoder transforms the system’s distributed cognitive state into a structured representation suitable for language generation. It comprises 20 specialized channels (Table 1), each encoding a distinct aspect of the cognitive state:

Table 1: ThoughtEncoder channels and their cognitive sources.

Ch.	Name	Source	Dimension
1–4	Perception	Sensory cortex regions	4×128
5–6	Memory	Episodic + semantic retrieval	2×256
7–8	Emotion	Neuromodulator bath state	2×128
9–10	Reasoning	Prefrontal workspace	2×256
11–12	Planning	Executive action plans	2×128
13–14	Social	Theory of Mind models	2×128
15–16	Ethical	Moral algebra verdicts	2×64
17–18	Epistemic	Knowledge confidence maps	2×256
19	Affective	Hedonic valence + arousal	1×128
20	Meta	Consciousness level + Φ	1×64

Each channel produces a sub-vector that is projected to a common dimensionality through learned linear transformations, then combined via HDC bundling to produce the composite thought vector $\mathbf{T} \in \{0, 1\}^D$ where $D = 16,384$.

3.3 HDC Binding Layer

The thought vector $\mathbf{T}$ is bound with positional and contextual information to produce a generation-ready representation:

$$\mathbf{G} = \mathbf{T} \otimes \rho^t(\mathbf{P}_{\text{context}}) \otimes \mathbf{K}_{\text{grounding}} \quad (1)$$

where $\mathbf{P}_{\text{context}}$ encodes the conversational context, ρ^t is a t -step permutation encoding temporal position, and $\mathbf{K}_{\text{grounding}}$ is an HDV representing the system’s grounded knowledge state. The binding with $\mathbf{K}_{\text{grounding}}$ is critical: it ensures that the generation vector $\mathbf{G}$ is structurally coupled to verified knowledge, such that similarity queries against $\mathbf{G}$ inherently favor epistemically supported content.

3.4 Epistemic Gating

The EpistemicGate is the central contribution of this paper. For each candidate token w in the vocabulary $\mathcal{V}$, the gate computes an epistemic support score:

$$\text{ES}(w) = \frac{D - d_H(\mathbf{G}, \mathbf{V}_w)}{D} \quad (2)$$

where $\mathbf{V}_w \in \{0, 1\}^D$ is the HDV representation of token w and d_H denotes Hamming distance. This normalized similarity score ranges from 0 (no support) to 1 (perfect match).

The gate then applies a consciousness-modulated threshold:

$$\tau(t) = \tau_{\text{base}} + \alpha \cdot \Phi(t) + \beta \cdot \text{LR}(t) \quad (3)$$

where $\tau_{\text{base}} = 0.52$ is the base threshold (slightly above chance similarity for random binary vectors), $\Phi(t)$ is the current consciousness level, $\text{LR}(t)$ is the current learning rate, and α, β are scaling coefficients. Higher consciousness increases the threshold, demanding stronger epistemic support for generation.

The logit masking operation is:

$$\tilde{\ell}(w) = \begin{cases} \ell(w) & \text{if } \text{ES}(w) \geq \tau(t) \\ -\infty & \text{otherwise} \end{cases} \quad (4)$$

where $\ell(w)$ is the raw logit for token w from the Liquid-Mamba backend. Tokens masked to $-\infty$ receive zero probability after softmax, physically preventing their selection.

Algorithm 1 Epistemic Gating for Token Generation

Require: Generation vector $\mathbf{G}$, vocabulary HDVs $\{\mathbf{V}_w\}_{w \in \mathcal{V}}$, raw logits $\{\ell(w)\}$, threshold τ

Ensure: Masked logits $\{\tilde{\ell}(w)\}$

```

1: for each  $w \in \mathcal{V}$  do
2:    $\text{ES}(w) \leftarrow (D - d_H(\mathbf{G}, \mathbf{V}_w))/D$ 
3:   if  $\text{ES}(w) < \tau$  then
4:      $\tilde{\ell}(w) \leftarrow -\infty$ 
5:   else
6:      $\tilde{\ell}(w) \leftarrow \ell(w)$ 
7:   end if
8: end for
9: return  $\{\tilde{\ell}(w)\}$ 

```

3.5 Coherence Feedback and Semantic Veto

The CoherenceFeedback module monitors the semantic trajectory of generated text and triggers mid-sentence correction when coherence degrades. After each generated token w_t , the module computes:

$$\text{Coherence}(t) = \frac{D - d_H(\mathbf{G}, \mathbf{S}_t)}{D} \quad (5)$$

where $\mathbf{S}_t = \bigoplus_{i=1}^t \rho^i(\mathbf{V}_{w_i})$ is the bundled sequence vector of all tokens generated so far. If $\text{Coherence}(t)$ drops below a threshold $\tau_c = 0.48$, a *semantic veto* is triggered: the current generation is abandoned, the sequence is rolled back to the last coherent checkpoint, and generation resumes with increased epistemic threshold $\tau' = \tau + 0.05$.

This mechanism implements a form of metacognitive monitoring for language production: the system “notices” when it is drifting from its intended meaning and self-corrects before completing the utterance.

3.6 Adaptive Cadence

Not every cognitive cycle warrants language generation. The adaptive cadence controller determines *when* to generate based on:

$$P(\text{generate} \mid t) = \sigma(w_c \cdot C(t) + w_l \cdot \text{LR}(t) + w_q \cdot Q_{\text{EMA}}(t) - b) \quad (6)$$

where $C(t)$ is the consciousness level, $\text{LR}(t)$ is the learning rate, $Q_{\text{EMA}}(t)$ is the exponential moving average of recent generation quality, σ is the logistic function, and w_c, w_l, w_q, b are learned parameters. This ensures that generation occurs more frequently during high-consciousness, high-learning states and less frequently during low-quality or low-consciousness periods.

3.7 Liquid-Mamba Fusion Backend

The autoregressive backbone combines Liquid Time-Constant (LTC) neurons [Hasani et al., 2021] with selective state space model (Mamba) layers [Gu and Dao, 2024]. The LTC component provides continuous-time dynamics:

$$\frac{d\mathbf{h}}{dt} = -\frac{1}{\tau(\mathbf{x})} (\mathbf{h} - f(\mathbf{x}, \mathbf{h})) \quad (7)$$

where $\tau(\mathbf{x})$ is an input-dependent time constant and f is a nonlinear function. The Mamba component provides efficient sequence modeling with selective scanning. The fusion architecture alternates LTC and Mamba layers, with the LTC layers providing temporal coherence and the Mamba layers providing efficient long-range dependencies.

4 Experimental Evaluation

4.1 Setup

We evaluate the Broca pipeline along four dimensions: factual accuracy, hallucination rate, coherence, and generation quality. The test suite comprises 358 automated tests covering unit, integration, and end-to-end scenarios.

4.2 Hallucination Prevention

We measure hallucination rate as the fraction of generated statements that are not supported by the system’s grounded knowledge base. Evaluation is performed on a set of 500 generation episodes across diverse topics (Table 2).

Table 2: Hallucination rates across generation conditions. “Baseline” uses raw logits without epistemic gating. “EG” adds the EpistemicGate. “EG+CF” adds CoherenceFeedback. “Full” includes adaptive cadence.

Condition	Halluc. Rate (%)	Coherence	Fluency	Tokens/s
Baseline	23.4	0.71	0.89	142
EG only	1.8	0.73	0.84	118
EG + CF	0.6	0.82	0.86	104
Full pipeline	0.4	0.84	0.87	89

The EpistemicGate alone reduces hallucination from 23.4% to 1.8%—a 92.3% reduction. Adding CoherenceFeedback further reduces it to 0.6%, and the full pipeline achieves 0.4%. The residual 0.4% consists primarily of compositional hallucinations where individually supported tokens combine to form unsupported claims, a limitation discussed in Section 5.5.

4.3 Epistemic Support Distribution

We analyze the distribution of epistemic support scores across the vocabulary during generation (Table 3).

Table 3: Epistemic support statistics during generation. $|\mathcal{V}_{\text{pass}}|$ denotes the number of tokens passing the epistemic gate.

Consciousness Level	Threshold τ	Mean ES	$ \mathcal{V}_{\text{pass}} $	Pass Rate (%)
Low ($\Phi < 0.10$)	0.525	0.507	4,218	13.2
Medium ($0.10 \leq \Phi < 0.15$)	0.540	0.512	3,156	9.9
High ($\Phi \geq 0.15$)	0.558	0.518	2,341	7.3

At higher consciousness levels, the threshold increases and fewer tokens pass the gate, reflecting the system’s heightened epistemic standards during conscious generation. The vocabulary is effectively pruned from $|\mathcal{V}| = 32,000$ to 2,000–4,000 tokens, concentrating probability mass on epistemically supported options.

4.4 Semantic Veto Frequency

The CoherenceFeedback module’s semantic veto is triggered when generation coherence drops below threshold. We characterize veto frequency and its relationship to generation quality (Table 4).

Table 4: Semantic veto statistics across 500 generation episodes.

Metric	Mean	Std	Range
Vetoes per episode	0.34	0.61	0–4
Tokens before veto	12.7	5.3	3–31
Post-veto quality improvement	+0.12	0.08	−0.03–+0.31
Rollback tokens	3.2	1.8	1–9

Vetoes occur in approximately 28% of episodes (mean 0.34 per episode), typically after 12–13 tokens. The quality of generation after rollback and resumption improves by a mean of 0.12 (on a 0–1 scale), confirming that the veto mechanism successfully redirects generation toward more coherent output.

4.5 Adaptive Cadence Behavior

The cadence controller modulates generation frequency based on cognitive state (Table 5).

Table 5: Generation cadence across cognitive states. “Cycles/gen” indicates the number of cognitive cycles between generation events.

Cognitive State	Cycles/gen (mean)	Quality (mean)	Halluc. Rate (%)
High consciousness + high LR	2.1	0.91	0.1
High consciousness + low LR	4.3	0.88	0.3
Low consciousness + high LR	5.7	0.79	0.8
Low consciousness + low LR	8.9	0.72	1.2

The cadence controller successfully allocates more generation bandwidth to high-consciousness, high-learning states where generation quality is highest and hallucination risk is lowest.

4.6 Comparison with Post-Hoc Methods

We compare the full Broca pipeline against representative post-hoc hallucination mitigation methods, applied to the same Liquid-Mamba backend without epistemic gating (Table 6).

Table 6: Comparison with post-hoc hallucination mitigation methods.

Method	Halluc. (%)	Coherence	Latency (ms)	Prevents?
None (baseline)	23.4	0.71	7.1	No
Temperature scaling	18.2	0.68	7.2	No
Nucleus sampling ($p = 0.9$)	16.7	0.72	7.3	No
Self-consistency ($k = 5$)	8.1	0.74	35.5	No
RAG (top-5 retrieval)	6.3	0.76	42.8	No
Broca (full pipeline)	0.4	0.84	11.2	Yes

The Broca pipeline achieves the lowest hallucination rate (0.4%) with competitive latency (11.2ms vs. 7.1ms baseline). Unlike all other methods, it *prevents* hallucination by construction rather than detecting it post-hoc. The latency overhead of 4.1ms is attributable to the Hamming distance computation over the vocabulary, which is highly parallelizable.

5 Discussion

5.1 Prevention versus Detection

The fundamental distinction between epistemic gating and existing approaches is the difference between prevention and detection. Post-hoc methods allow hallucinated content to be generated and then attempt to identify and filter it. This creates an adversarial dynamic: as language models become more fluent, hallucinated content becomes harder to distinguish from factual content, and post-hoc detection becomes less reliable [Ji et al., 2023].

Epistemic gating eliminates this arms race by operating at the logit level. Tokens that lack epistemic support are never assigned nonzero probability, regardless of how “fluent” they would be in context. The system cannot generate what it does not know, just as a biological speaker cannot articulate concepts that have not achieved conscious access [Dehaene, 2014].

5.2 Consciousness-Awareness as a Feature

The modulation of the epistemic threshold by consciousness level (Φ) implements a form of metacognitive regulation. During high-consciousness states—when the system’s integrated information is high and its internal representations are most coherent—the threshold increases, demanding stronger epistemic support. During low-consciousness states, the threshold relaxes, permitting more exploratory generation.

This mirrors the human experience of verbal fluency varying with attentional state. Under high cognitive load or reduced consciousness (fatigue, distraction), speech becomes less precise and more error-prone [Levelt, 1989]. Our system formalizes this relationship: consciousness level directly modulates the stringency of epistemic filtering.

5.3 HDC as Epistemic Infrastructure

The use of HDC for epistemic support computation is not incidental. The high dimensionality ($D = 16,384$) provides a rich similarity space where graded epistemic relationships are naturally encoded. The quasi-orthogonality of random HDVs ensures that the baseline similarity is approximately 0.50 (for binary vectors), providing a natural chance level against which epistemic support is measured.

The binding operation $\mathbf{G} = \mathbf{T} \otimes \rho^t(\mathbf{P}) \otimes \mathbf{K}$ creates a holographic representation that simultaneously encodes the thought content, contextual position, and knowledge grounding. Similarity to this composite vector requires a token to be consistent with all three factors, providing a more robust epistemic check than any single factor alone.

5.4 Active Inference Interpretation

Under the active inference framework [Friston, 2010], language generation is an epistemic action that reduces the system’s uncertainty about its own beliefs. The Epistemic-Gate can be interpreted as a precision-weighting mechanism: tokens with high epistemic support have high precision (low uncertainty), while tokens with low support have low precision and are suppressed.

The adaptive cadence controller implements a form of expected free energy minimization: the system generates language when doing so is expected to reduce free energy (high consciousness, high learning rate) and withholds generation when the expected information gain is low.

5.5 Limitations

Several limitations warrant discussion. First, the residual 0.4% hallucination rate arises from *compositional* hallucinations: individual tokens may be epistemically supported, but their combination produces an unsupported claim. For example, the tokens “the,” “president,” “lives,” “on,” “Mars” are individually supported, but their composition is not.

Addressing compositional hallucination requires extending the epistemic check to n -gram or phrase-level representations, which we leave to future work.

Second, the epistemic gate may be overly conservative, suppressing novel but valid inferences. The system can only generate claims directly supported by its knowledge base, which limits creative and abductive reasoning. This is a fundamental trade-off between safety and capability.

Third, the computational overhead of per-token Hamming distance computation scales as $O(|\mathcal{V}| \cdot D)$ per generation step. For our vocabulary of 32,000 tokens and $D = 16,384$, this amounts to approximately 5×10^8 bit operations per step, which is tractable on modern hardware but nontrivial.

Fourth, our evaluation is conducted within the Broca pipeline’s own knowledge representation. External validation against human-annotated factuality benchmarks (e.g., TruthfulQA [Lin et al., 2022]) would strengthen the empirical claims, and we plan this evaluation in future work.

5.6 Relationship to Neuroscience

Our architecture is inspired by, but does not claim to replicate, the neural mechanisms of language production. In the human brain, Broca’s area (left inferior frontal gyrus) is involved in syntactic processing, phonological encoding, and the sequencing of articulatory gestures [Hagoort, 2005]. Our Broca pipeline abstracts these functions into the ThoughtEncoder (conceptual encoding), HDC binding (structural encoding), and autoregressive generation (sequential output).

The epistemic gating mechanism has no direct neural analog that we are aware of, though it is functionally similar to the “editor” or “monitor” proposed in speech production models [Levelt, 1989], which filters pre-articulatory representations for errors before they reach motor execution.

6 Related Work

Ji et al. [2023] provide the most comprehensive survey of hallucination in language models, categorizing causes and mitigation strategies. Our work differs in proposing prevention rather than mitigation.

Lewis et al. [2020] introduced retrieval-augmented generation, which grounds generation in retrieved documents. RAG and epistemic gating are complementary: RAG provides external knowledge grounding, while epistemic gating provides internal epistemic filtering. A combined system could use RAG to populate the knowledge HDVs that the EpistemicGate queries.

Kadavath et al. [2022] showed that language models can be trained to express calibrated uncertainty. Our approach differs in that uncertainty is computed from HDC similarity rather than learned calibration, providing a more interpretable and verifiable uncertainty measure.

Kanerva [2009] established the theoretical foundations of HDC. Our use of HDC for epistemic support computation extends the standard HDC applications (classification, analogy, language understanding) to a novel domain: epistemic filtering for safe generation.

7 Conclusion

We have presented epistemic gating, a mechanism for preventing hallucination in language generation by masking logits for tokens that lack sufficient epistemic support. Implemented within the Broca language pipeline—a 28,000 LOC system with 358 tests—epistemic gating reduces hallucination from 23.4% to 0.4%, a 98.3% reduction. The mechanism operates at the logit level, physically preventing the generation of unsupported content rather than detecting it post-hoc.

The key architectural innovations are: (1) encoding cognitive state as a 16,384-dimensional HDV via a 20-channel ThoughtEncoder; (2) computing per-token epistemic support via HDC similarity; (3) consciousness-modulated threshold adjustment; and (4) mid-sentence semantic veto for coherence maintenance. Together, these mechanisms implement a consciousness-aware language generation system that can only articulate what it knows.

Future work will address compositional hallucination via phrase-level epistemic checking, external benchmark evaluation, and the integration of retrieval-augmented knowledge into the HDC grounding layer. We also plan to investigate the theoretical connections between epistemic gating and the free energy principle more formally, potentially deriving the gating threshold from variational inference principles.

8 Reproducibility

All experiments were conducted using the Broca language pipeline within the Symthaea cognitive architecture (v1.9.0). The following commands reproduce the core results:

```
# Run Broca pipeline tests (358 tests)
cargo test -p symthaea-broca -- --nocapture

# Run epistemic gating unit tests
cargo test -p symthaea-broca --lib epistemic -- --nocapture

# Run coherence feedback tests
cargo test -p symthaea-broca --lib coherence -- --nocapture

# Run end-to-end generation tests
cargo test -p symthaea-broca --lib e2e -- --nocapture

# Build with language feature enabled
cargo build --features ssm_language
```

The system requires Rust 1.75+ and the `nix develop` environment. The `ssm_language` feature flag must be enabled for Broca pipeline functionality. The Liquid-Mamba backend requires the default feature set.

Code Availability

The Broca language pipeline source code is located in `symthaea/crates/symthaea-broca/` within the Luminous Dynamics Symthaea repository. The `EpistemicGate` implementation, `CoherenceFeedback` module, and adaptive cadence controller are all contained within

this crate. Correspondence and access requests should be directed to the corresponding author.

References

- Baars, B. J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.
- Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.
- Bender, E. M., Gebru, T., McMillan-Major, A., and Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *FAccT '21*, pp. 610–623.
- Dehaene, S. (2014). *Consciousness and the Brain*. Viking Press.
- Fedorenko, E. and Thompson-Schill, S. L. (2014). Reworking the language network. *Trends in Cognitive Sciences*, 18(3):120–126.
- Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138.
- Gu, A. and Dao, T. (2024). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Hagoort, P. (2005). On Broca, brain, and binding: a new framework. *Trends in Cognitive Sciences*, 9(9):416–423.
- Hasani, R., Lechner, M., Amini, A., et al. (2021). Liquid time-constant networks. In *AAAI '21*, pp. 7657–7666.
- Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Kadavath, S., Conerly, T., Aspell, A., et al. (2022). Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.
- Kanerva, P. (2009). Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1(2):139–159.
- Levelt, W. J. (1989). *Speaking: From Intention to Articulation*. MIT Press.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS 2020*.
- Lin, S., Hilton, J., and Evans, O. (2022). TruthfulQA: Measuring how models mimic human falsehoods. In *ACL 2022*, pp. 3214–3252.
- Maynez, J., Narayan, S., Bohnet, B., and McDonald, R. (2020). On faithfulness and factuality in abstractive summarization. In *ACL 2020*, pp. 1906–1919.
- Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. In *NeurIPS 2022*.

- Wang, X., Wei, J., Schuurmans, D., et al. (2023). Self-consistency improves chain of thought reasoning in language models. In *ICLR 2023*.
- Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS 2022*.
- Thorne, J., Vlachos, A., Christodoulopoulos, C., and Mittal, A. (2018). FEVER: a large-scale dataset for fact extraction and VERification. In *NAACL 2018*, pp. 809–819.
- Tononi, G. (2004). An information integration theory of consciousness. *BMC Neuroscience*, 5(1):42.